

#4. Tabular Representation Learning

Instructor: **Hankook Lee** @ Efficient Learning Lab., Sungkyunkwan University

Lecture Overview

- This lecture covers **representation learning** for tabular data
 - **Why representation learning?**
 - **SSL:** How to learn useful representations from unlabeled tables?
 - VIME (2020), SAINT (2021), SubTab (2021), SCARF (2022), RaTab (2025), T-JEPA (2025)
 - **Few-Shot & Transferable Representations:** Can learned representations transfer to low-label or cross-table settings?
 - STUNT (2023), TransTab (2022), CARTE (2024)
 - **Anomaly Detection:** How can representation learning help model normal patterns and detect abnormal samples?
 - ICL (2022), MCM (2024), NPT-AD (2024), Disent-AD (2025), DRL (2025)

Why Representation Learning?

- **Motivation:** Learn from limited labels and abundant unlabeled data
 - Labels are often expensive, scarce, or noisy
 - Unlabeled tables can be abundant and easier to collect
- **Challenge:** Learning meaningful representations from tables
 - Heterogeneous features: numerical, categorical, binary, and missing values
 - No universal spatial or sequential structure unlike images or text
 - Diverse domains with domain-specific semantics
 - No universal augmentation strategy for tabular data

How can we **learn useful representations from unlabeled tables** while preserving their semantics?

Applications

- **Self-supervised prediction & pretraining**
 - Learn representations from unlabeled rows, then fine-tune with labels
- **Few-shot & transferable learning**
 - Reuse learned representations when labels are scarce or tables differ
- **Anomaly detection**
 - Learn normal patterns and identify samples that deviate from them
- **Semantic and foundation-model extensions**
 - Incorporate metadata, text descriptions, or broader context in later lectures

Learning from Unlabeled Tables

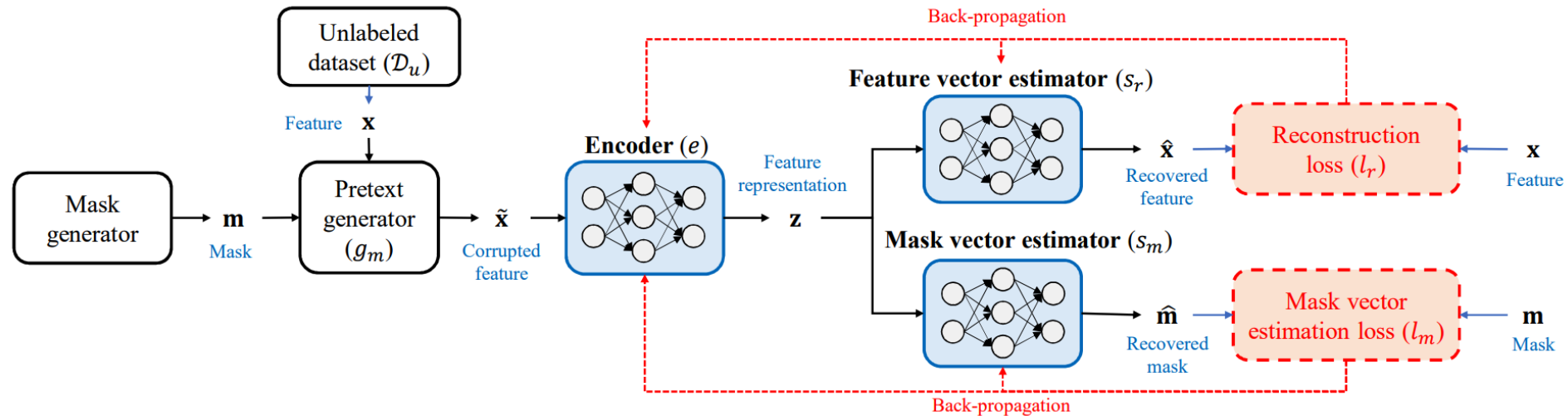
- **Self-supervised learning** learns representations without task labels
 - Create a **pretext task** from unlabeled data
 - Pre-train an encoder using the pretext task
 - Fine-tune the learned representation for downstream prediction
- The representation **quality depends on the design of the pretext task**

Key Question

What self-supervised task is appropriate for tabular data?

VIME: Reconstruction-Based SSL

- **Key Idea:** Corrupt tabular inputs and train the model to recover
 - **Randomly mask** some features & replace masked values by other rows
 - **Two Pretext Tasks**
 - **Mask estimation:** Which features were corrupted?
 - **Value imputation:** What were the original feature values?



VIME learns representations by identifying and restoring corrupted feature values

VIME: Intuition and Limitation

- **Why can it work?**

- To reconstruct missing or corrupted features, **the model must learn dependencies among columns**
- E.g., occupation, income, and education may help predict one another

- **Practical extension**

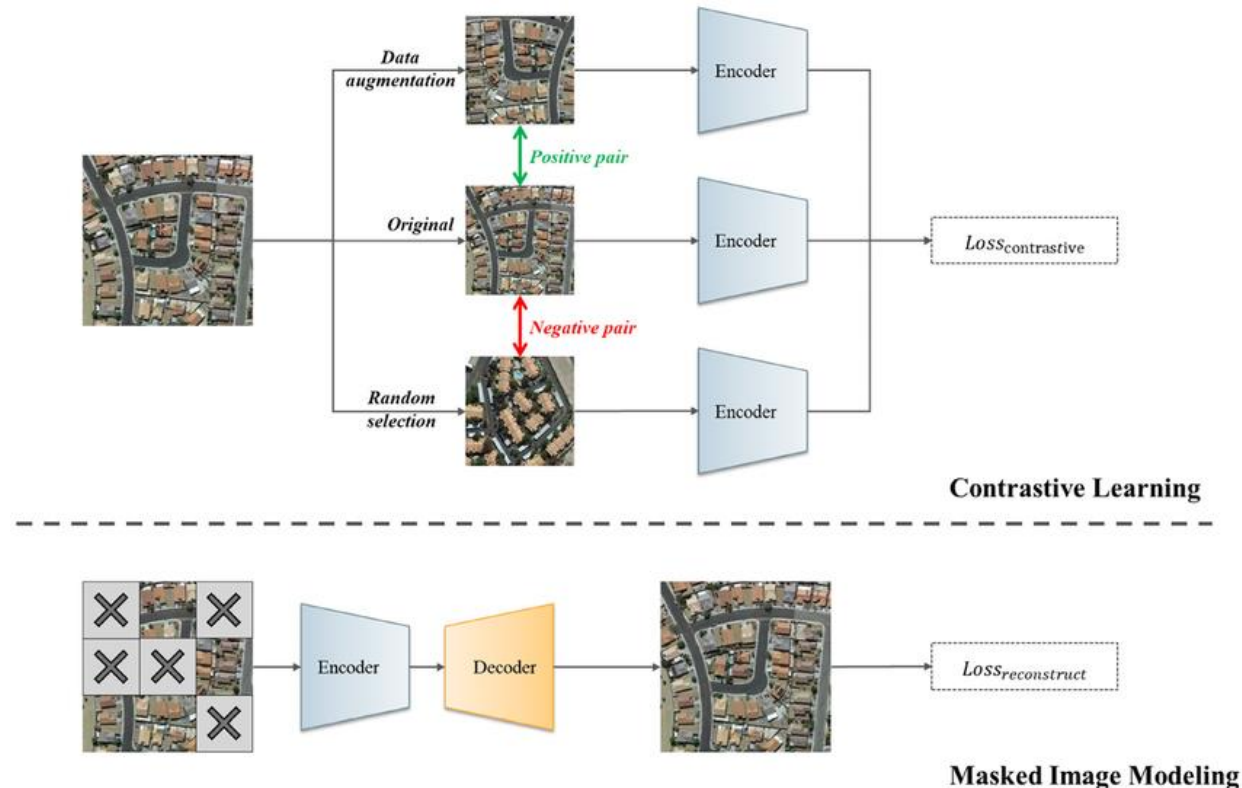
- VIME also uses unlabeled samples in a semi-supervised learning framework

- **Limitations**

- Depends on whether corruption creates a meaningful learning task
- Arbitrary value replacement may produce unrealistic samples

From Recon. to Contrastive Learning

- **Reconstruction vs Contrastive Learning:**
 - **Reconstruction:** Can the model recover corrupted inputs?
 - **Contrastive learning:** Can the model recognize two related views?

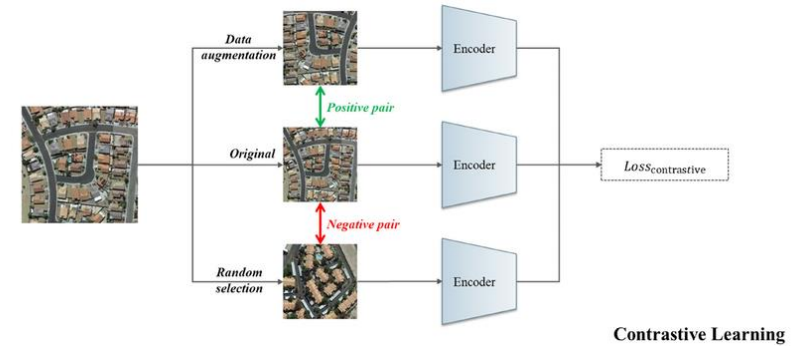


From Recon. to Contrastive Learning

- **Reconstruction vs Contrastive Learning:**
 - **Reconstruction:** Can the model recover corrupted inputs?
 - **Contrastive learning:** Can the model recognize two related views?

- **Goal of Contrastive Learning**

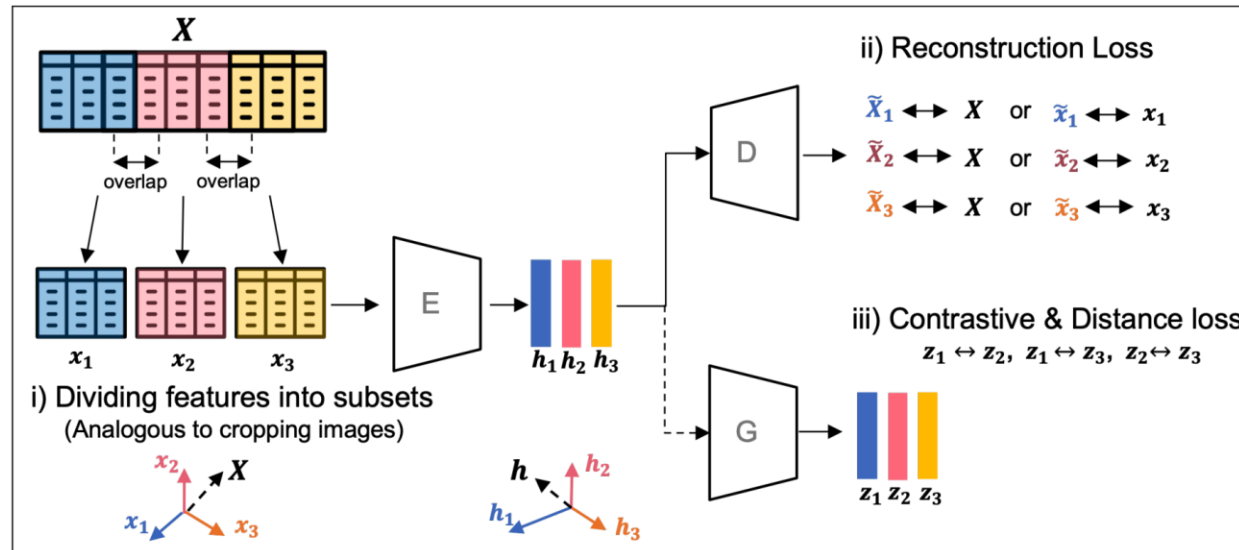
- Pull together representations of related views
- Separate representations of different samples



- **Key Question:** How should two views of a tabular row be generated?
 - This view construction is the central problem for tabular contrastive learning

SubTab: Views from Feature Subsets

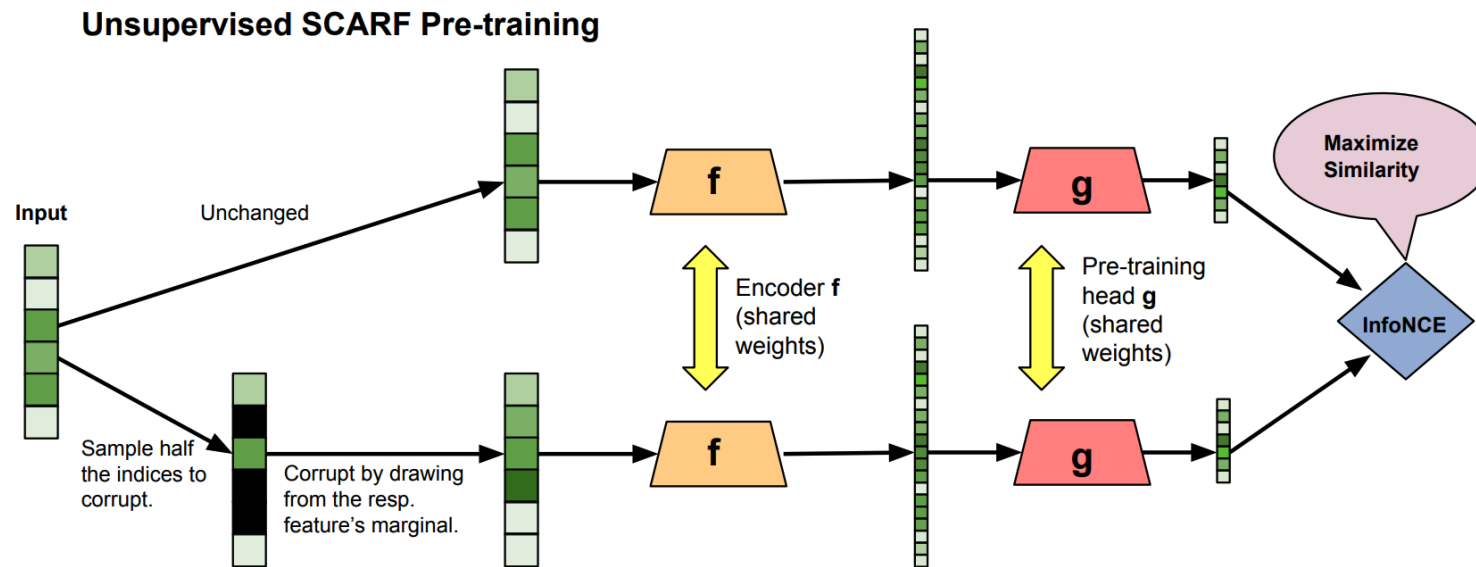
- **Key Idea:** Treat different subsets of columns as multiple views of the same row
 - Each subset represents partial information about the same sample
 - Learn representations that are consistent across subsets



Useful representations should capture information shared across partial views

SCARF: Views from Feature Corruption

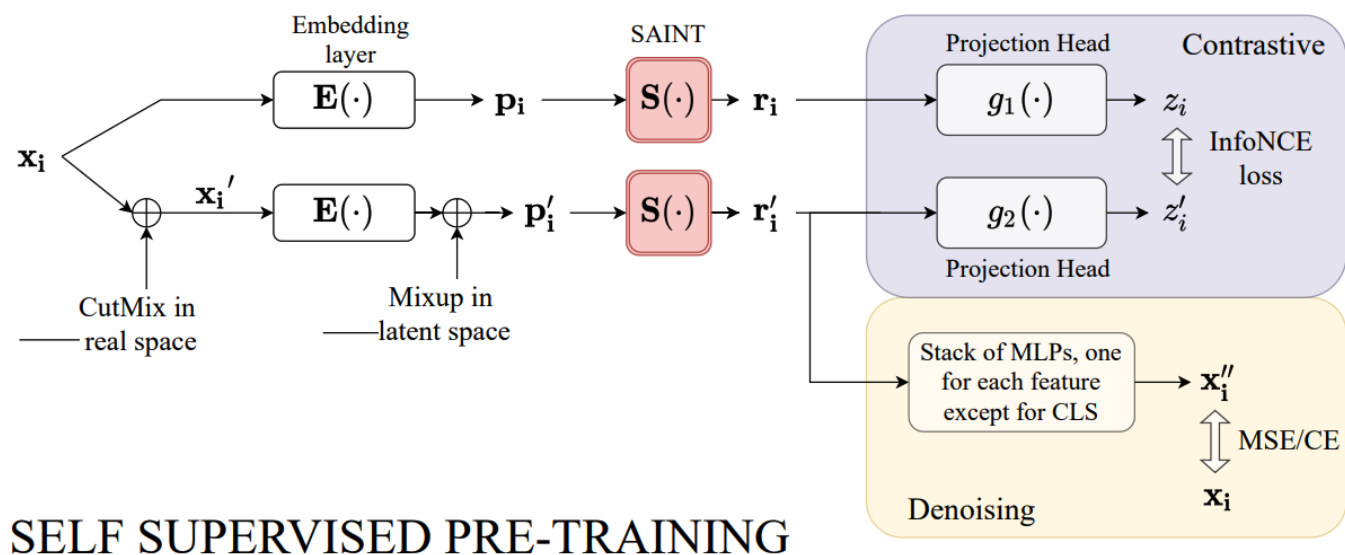
- **Key Idea:** Create a positive view by randomly corrupting a subset of features
 - Replace a feature with values sampled **from their marginal distributions**
 - Treat the original and corrupted rows as a positive pair



A simple and general strategy for contrastive tabular pre-training

SAINT: Views from CutMix & Mixup

- **Key Idea:** Create a positive view by CutMix & Mixup augmentations
 - Use **CutMix-style feature replacement** to mix feature values across samples
 - Use **Mixup-style interpolation** in the embedding space
 - **Additional contribution:** architecture with intersample attention



CutMix and Mixup-style augmentations for tabular SSL

What Have We Learned So Far?

- Why **combine** reconstruction and contrastive learning?
 - Reconstruction encourages the model to learn feature dependencies
 - Contrastive learning encourages stable representations across different views
 - Both are useful because each feature may carry task-critical semantics
- Early tabular SSL methods **rely on manually-designed input-level augmentations**
 - E.g., corruption, mask, replace, mix, or subset features
 - **Common Assumption:** Modified views should preserve the essential meaning of the original row

Beyond Input-Level Augmentations

- **Why Move Beyond Input-Level Augmentation?**

- Input-level augmentation remains difficult for tabular data
 - Removing features may discard task-relevant information
 - Replacing values may create unrealistic rows
 - Valid transformations vary across domain

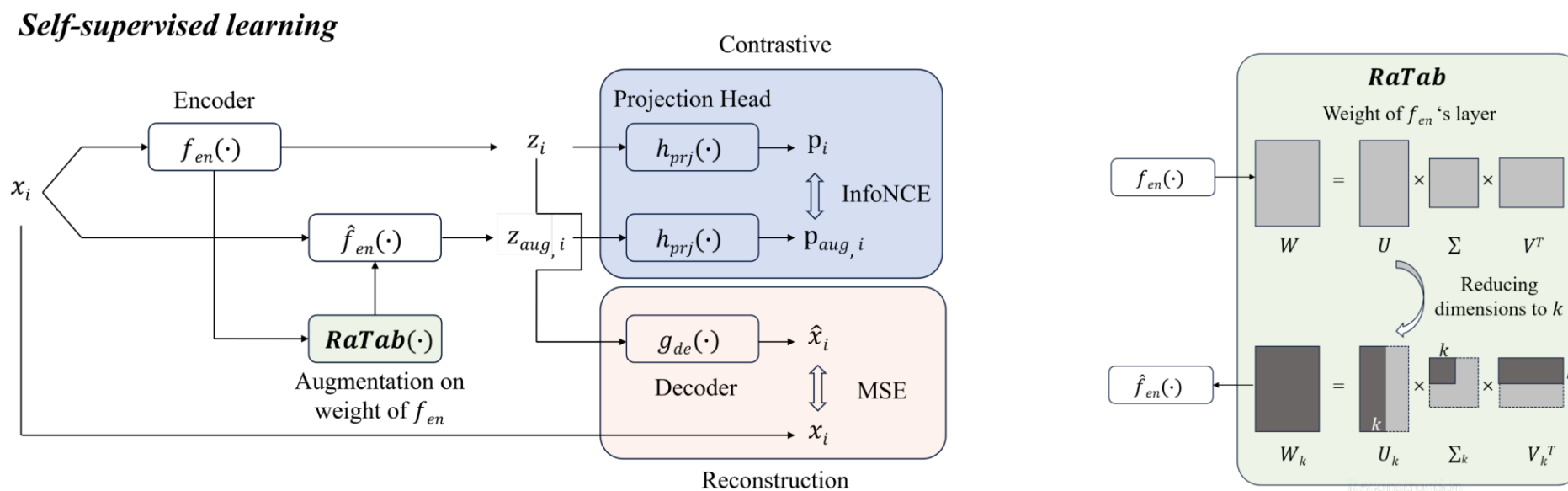
Recent tabular SSL methods seek learning signals that are less dependent on hand-designed input corruption

- **Recent Direction:**

- Do not directly modify raw feature values
- Instead, manipulate or predict representations in latent space

RaTab: Augmentation in Latent Space

- **Key Idea:** Modify the encoder to generate alternative views instead of modifying raw feature values
 - Apply truncated SVD to the final-layer weight matrix of the encoder
 - Construct an augmented encoder using the truncated weights
 - Use both **reconstruction** and **contrastive** losses



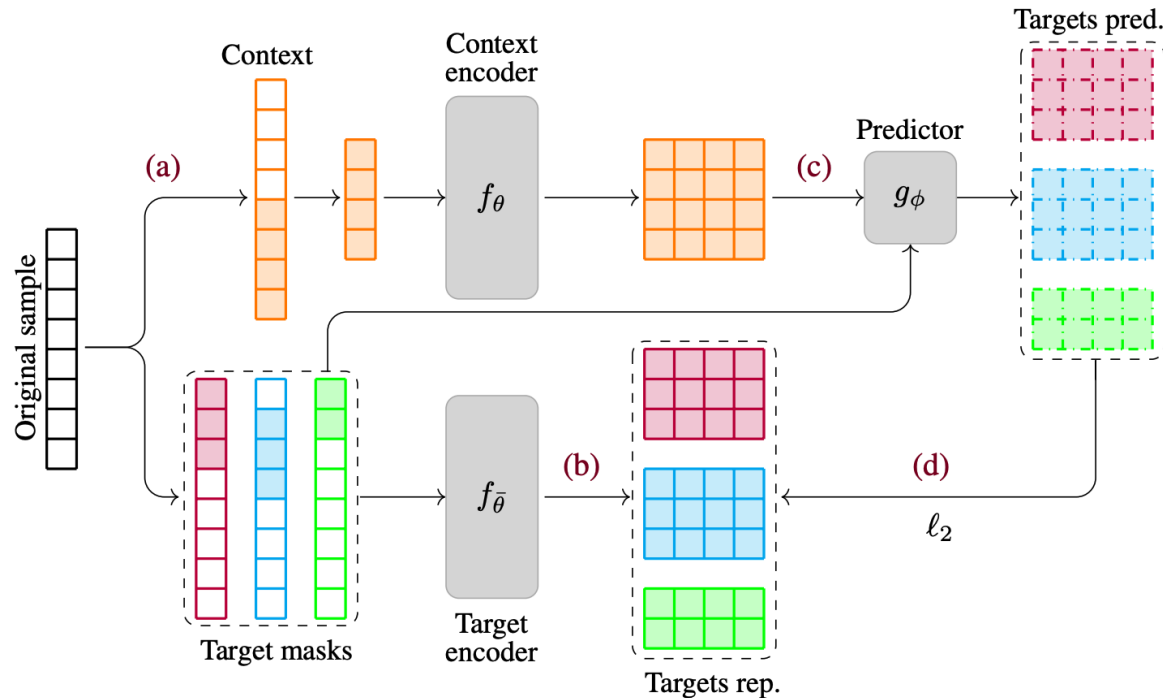
RaTab: Augmentation in Latent Space

- RaTab pretraining **improves performance of various architectures**
 - Evaluation protocol: RaTab pretraining → supervised fine-tuning

Method	Binary						Multiclass			Regression			
	CH ↑	AD ↑	BM ↑	CS ↑	HI ↑	PO ↑	GE ↑	HE ↑	OT ↑	CPU ↓	DIA ↓	FI ↓	HO ↓
<i>GBDT models</i>													
XGBoost	0.858	<u>0.871</u>	0.813	0.730	0.724	0.876	0.684	0.368	0.808	3.201	856.366	11131.716	3.169
CatBoost	0.855	<u>0.869</u>	0.815	0.760	<u>0.737</u>	0.904	0.729	0.381	0.819	2.607	849.638	9626.546	4.352
LightGBM	0.869	0.868	0.816	0.710	<u>0.731</u>	0.896	0.706	0.368	<u>0.823</u>	2.811	843.905	10622.170	4.519
<i>DNN models</i>													
MLP	0.858	0.849	0.792	0.680	0.720	0.906	0.586	0.375	0.804	2.925	868.154	10754.350	3.173
FT-Transformer	0.867	0.859	0.805	0.730	0.731	<u>0.909</u>	0.690	0.391	0.803	2.791	837.665	9567.579	3.124
ResNet	0.858	0.854	0.802	0.720	0.731	<u>0.880</u>	0.681	0.379	0.806	2.869	853.437	10270.991	4.733
NODE	0.859	0.858	-	-	0.725	-	0.539	0.353	0.803	-	-	-	3.216
TabNet	0.850	0.850	-	-	0.720	-	0.600	0.379	0.791	-	-	-	3.252
DCNv2	0.857	0.853	-	-	0.722	-	0.557	0.385	0.802	-	-	-	3.172
T2G-former	0.863	0.862	0.798	0.730	0.734	0.887	0.656	0.391	0.819	2.878	851.951	10121.356	3.138
SAINT	<u>0.885</u>	0.863	0.807	0.737	0.728	0.866	0.694	<u>0.392</u>	0.825	3.069	794.738	19366.582	5.652
SCARF	0.858	0.842	0.878	0.720	0.653	0.830	0.485	0.197	0.604	3.266	1091.008	9096.269	4.126
RaTab (on MLP)	0.881	0.865	<u>0.907</u>	0.820	0.722	0.924	0.694	0.384	0.813	2.772	527.028	8451.140	2.763
RaTab (on FT-Transformer)	0.880	0.868	<u>0.911</u>	0.790	<u>0.737</u>	0.898	0.676	<u>0.392</u>	0.805	2.588	498.598	8715.532	2.934
RaTab (on T2G-former)	0.888	0.873	0.912	<u>0.790</u>	0.738	0.907	0.753	0.397	0.826	2.489	<u>511.783</u>	7833.063	<u>2.856</u>

T-JEPA: Augmentation-Free Tabular SSL

- **Key Idea:** Predict one latent feature view from another, rather than reconstructing corrupted inputs
 - Predict a target feature subset from a context subset in latent space
 - Use feature masking only; no input-level augmentation required



T-JEPA: Augmentation-Free Tabular SSL

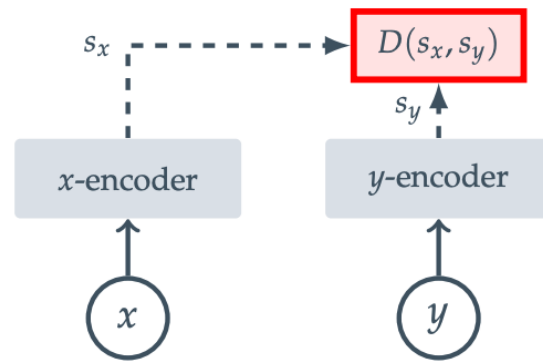
- **T-JEPA learns useful representations** for tabular prediction
 - Evaluation protocol: T-JEPA pretraining → training on learned representations

	AD ↑	HE ↑	JA ↑	AL ↑	CA ↓	HI ↑	MNIST ↑
Baseline Neural Networks							
MLP	0.827 $\pm 1e^{-3}$	0.353 $\pm 1e^{-3}$	0.672 $\pm 1e^{-3}$	0.916 $\pm 4e^{-3}$	0.511 $\pm 3e^{-3}$	0.681 $\pm 4e^{-3}$	0.978 $\pm 2e^{-3}$
+T-JEPA	0.866 $\pm 2e^{-3}$	0.400 $\pm 4e^{-3}$	0.728 $\pm 3e^{-3}$	0.961 $\pm 6e^{-3}$	0.468 $\pm 4e^{-2}$	0.517 $\pm 8e^{-2}$	0.983 $\pm 1e^{-3}$
DCNv2	0.829 $\pm 4e^{-3}$	0.347 $\pm 3e^{-3}$	0.662 $\pm 3e^{-3}$	0.905 $\pm 3e^{-3}$	0.504 $\pm 4e^{-3}$	0.683 $\pm 3e^{-3}$	0.971 $\pm 1e^{-3}$
+T-JEPA	0.861 $\pm 2e^{-3}$	0.399 $\pm 3e^{-3}$	0.723 $\pm 2e^{-3}$	0.955 $\pm 2e^{-3}$	0.420 $\pm 3e^{-2}$	0.525 $\pm 8e^{-2}$	0.981 $\pm 2e^{-3}$
ResNet	0.814 $\pm 7e^{-3}$	0.351 $\pm 2e^{-3}$	0.666 $\pm 3e^{-3}$	0.919 $\pm 2e^{-3}$	0.534 $\pm 2e^{-3}$	0.674 $\pm 4e^{-3}$	0.979 $\pm 1e^{-3}$
+T-JEPA	0.865 $\pm 3e^{-3}$	0.401 $\pm 2e^{-3}$	0.718 $\pm 3e^{-3}$	0.964 $\pm 1e^{-3}$	0.441 $\pm 8e^{-2}$	0.705 $\pm 5e^{-3}$	0.983 $\pm 2e^{-3}$
AutoInt	0.823 $\pm 1e^{-3}$	0.338 $\pm 3e^{-3}$	0.653 $\pm 6e^{-3}$	0.894 $\pm 2e^{-3}$	0.501 $\pm 3e^{-3}$	0.694 $\pm 3e^{-3}$	0.901 $\pm 6e^{-3}$
+T-JEPA	0.866 $\pm 2e^{-3}$	0.351 $\pm 3e^{-3}$	0.710 $\pm 3e^{-3}$	0.938 $\pm 4e^{-3}$	0.448 $\pm 2e^{-2}$	0.517 $\pm 8e^{-2}$	0.978 $\pm 1e^{-3}$
FT-Trans	0.821 $\pm 7e^{-3}$	0.363 $\pm 2e^{-3}$	0.677 $\pm 2e^{-3}$	0.913 $\pm 3e^{-3}$	0.473 $\pm 5e^{-3}$	0.684 $\pm 5e^{-3}$	0.811 $\pm 5e^{-2}$
+T-JEPA	0.864 $\pm 1e^{-3}$	0.384 $\pm 5e^{-3}$	0.708 $\pm 5e^{-3}$	0.921 $\pm 1e^{-2}$	0.444 $\pm 1e^{-1}$	0.551 $\pm 6e^{-2}$	0.966 $\pm 2e^{-3}$
Gradient Boosted Decision Trees (GBDT)							
XGBoost	<u>0.872</u> $\pm 5e^{-4}$	0.375 $\pm 1.2e^{-3}$	0.721 $\pm 1e^{-3}$	0.951 $\pm 1e^{-3}$	0.433 $\pm 2e^{-3}$	<u>0.729</u> $\pm 1e^{-3}$	0.980 $\pm 1e^{-3}$
CatBoost	0.873 $\pm 1e^{-3}$	0.381 $\pm 1e^{-3}$	0.721 $\pm 1e^{-3}$	0.946 $\pm 9e^{-4}$	0.430 $\pm 7e^{-4}$	0.726 $\pm 8e^{-4}$	0.972 $\pm 3e^{-3}$

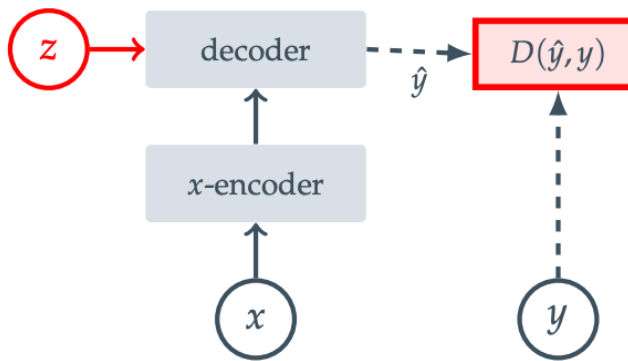
+ What is JEPA?

- **Joint-Embedding Predictive Architecture (JEPA)**

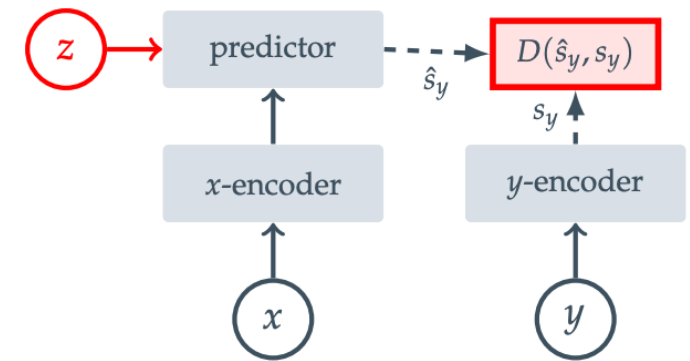
- A self-supervised learning framework that **learns to predict (or generate) latent in embedding space**, not input space
- JEPAs **do not seek representations invariant to a set of hand-crafted data augmentations** in contrast to Joint-Embedding Architectures



(a) Joint-Embedding Architecture



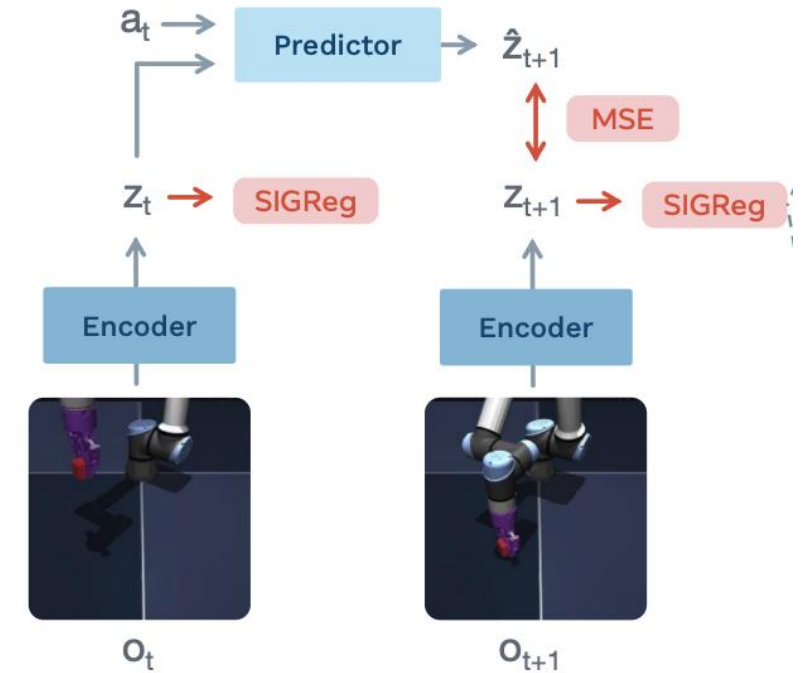
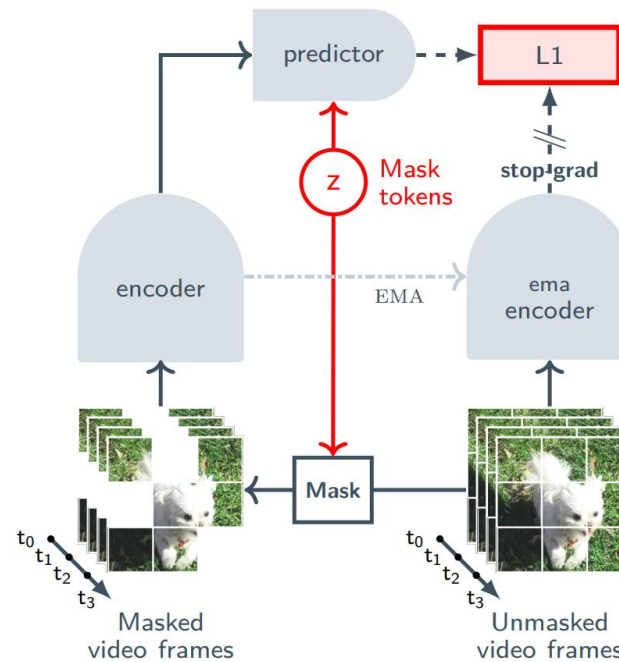
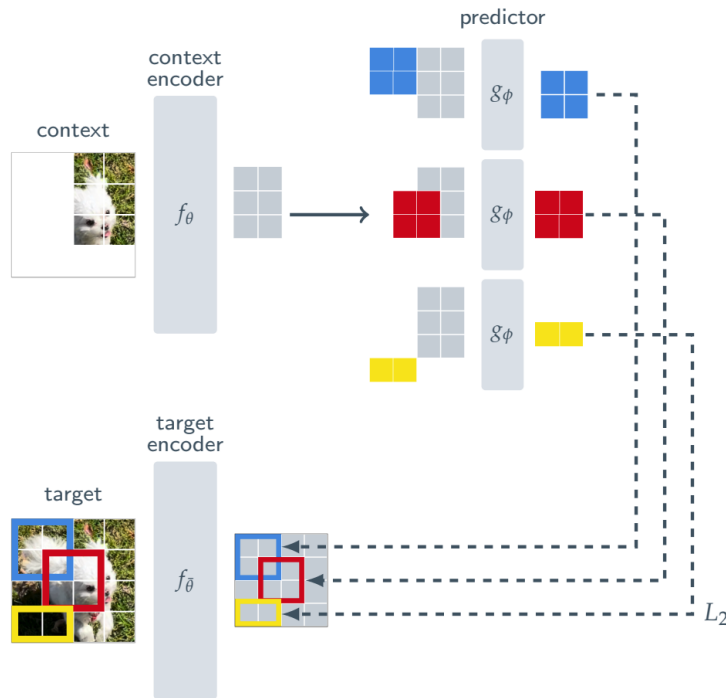
(b) Generative Architecture



(c) Joint-Embedding Predictive Architecture

+ What is JEPA?

- **Joint-Embedding Predictive Architecture (JEPA)**
 - Recently, JEPA has been widely applied across various domains
 - Image (I-JEPA), video (V-JEPA), world model (LeWorldModel), ...



Summary: SSL for Tabular Prediction

- **Key Question:** How can we learn useful representations from unlabeled tabular data?
 - How to capture meaningful dependencies among heterogeneous features?
 - How to design general methods without relying on domain-specific knowledge or hand-crafted augmentations?
- **Remaining Challenges**
 - No universally dominant SSL objective
 - Limited availability of large, diverse tabular datasets for general pretraining
 - Pretraining benefits may vary substantially across downstream datasets

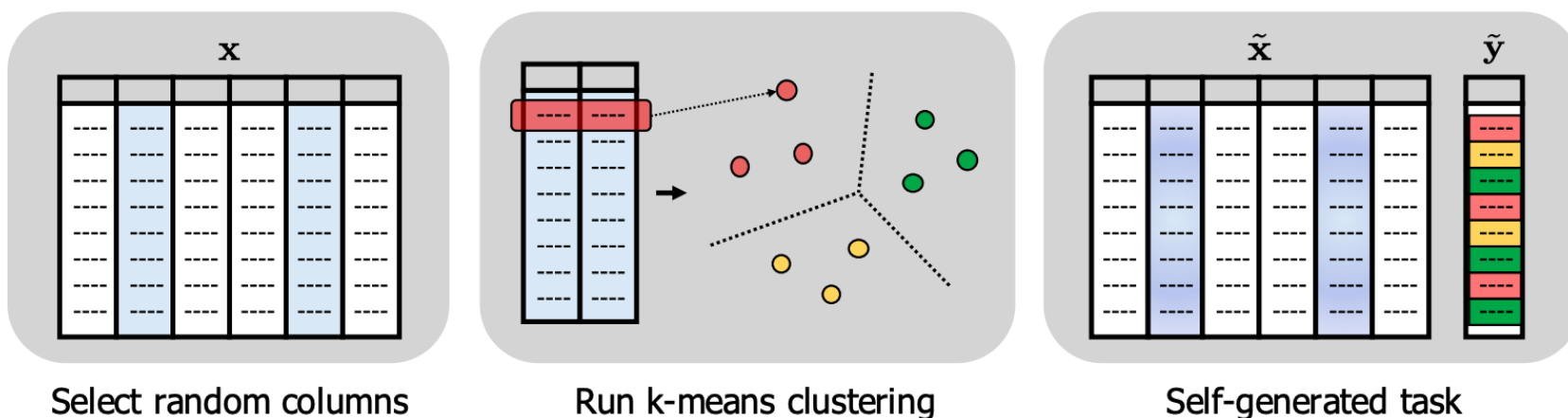
From SSL to Transferable Representations

- **SSL for prediction** usually assumes one target table
 - Pre-train on unlabeled rows
 - Fine-tune on labeled rows from the same dataset
- **Transferable tabular representations** relax this setting:
 - Learn from unlabeled tables
 - Adapt **with few labels**
 - Transfer **across different column structures**

The challenge shifts from learning within one table
to transferring across tasks or schemas

STUNT: Self-Generated Tasks

- **Key Idea:** Generate diverse pseudo-tasks from an unlabeled table by treating the table's column feature as a useful target
 - Randomly select a subset of columns
 - Cluster samples using the selected columns
 - Use the cluster assignments as pseudo-labels



- **Train a few-shot learner** across the diverse "self-generated" tasks

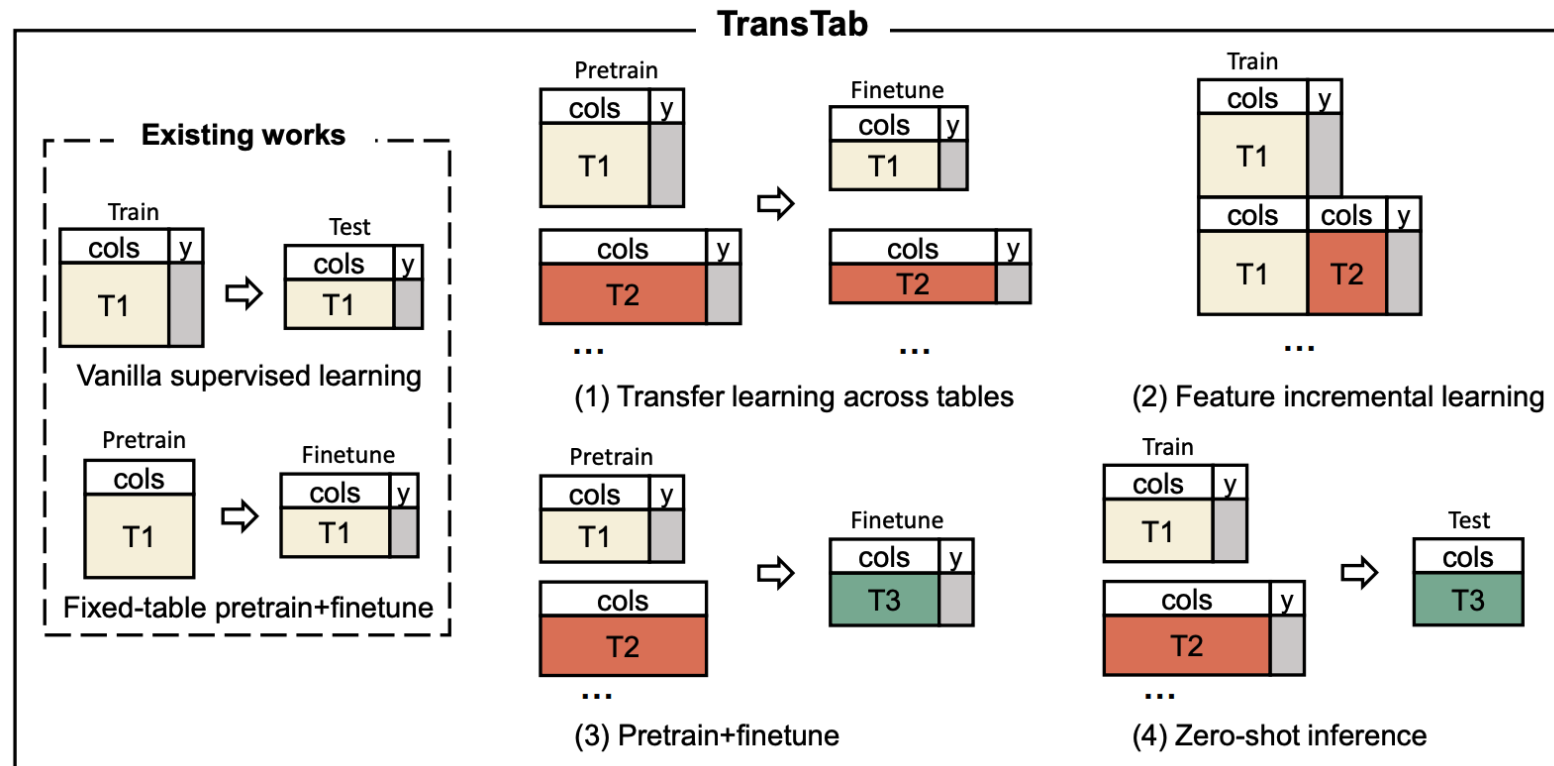
STUNT: Self-Generated Tasks

- STUNT can adapt to a task with only a few labeled samples
 - In such scenarios, supervised learning and SSL methods often fail because they do not have adaptation ability from few labeled samples

Type	Method	Val.	income	cmc	karhunen	optdigit	diabetes	semeion	pixel	dna	Avg.
# shot = 1											
Sup.	CatBoost	✓	57.00	34.60	55.67	61.32	60.02	43.21	59.16	41.35	52.06
	MLP	✓	60.52	35.06	48.67	61.02	57.25	40.88	55.62	44.39	50.43
	LR	✓	59.64	35.08	55.05	65.19	57.61	42.90	59.71	44.28	52.43
	kNN	-	61.22	34.99	54.42	65.58	58.56	44.35	61.48	42.67	52.82
Semi-sup.	Mean Teacher	✓	60.63	35.58	54.57	66.10	58.05	43.56	61.02	46.58	53.26
	ICT	✓	61.83	36.53	58.37	69.12	58.08	43.48	60.88	46.55	54.36
	Pseudo-Label	✓	60.52	34.97	49.44	61.50	57.03	41.42	56.12	44.26	50.66
	MPL	✓	60.85	35.13	47.66	61.52	57.39	41.82	56.01	44.22	50.58
	VIME-Semi	✓	56.40	32.97	57.40	66.85	58.16	40.43	52.86	39.18	50.53
Self-sup.	SubTab + Fine-tune	✓	59.74	35.65	41.11	49.88	59.35	30.49	42.23	40.86	44.91
	SubTab + LR	✓	61.88	35.68	50.32	67.05	58.06	40.27	60.40	45.68	52.42
	SubTab + kNN	-	61.58	35.87	48.74	66.05	59.22	39.99	61.30	44.16	52.36
	VIME + Fine-tune	✓	60.50	34.98	47.50	61.31	57.23	41.09	53.79	44.30	50.09
	VIME + LR	✓	61.99	35.30	59.62	70.52	56.95	47.20	64.17	51.36	55.89
	VIME + kNN	-	62.16	35.55	58.56	69.31	58.35	46.99	64.62	50.29	55.78
Unsup.-Meta.	UMTRA	-	57.23	35.46	49.05	49.87	57.64	26.33	34.26	25.13	41.87
	SES	-	56.39	34.59	49.19	56.30	59.97	33.73	49.19	39.56	47.37
	CACTUs	-	64.02	36.10	65.59	71.98	58.92	48.96	67.61	65.93	59.89
	STUNT (Ours)	-	63.52	37.10	71.20	76.94	61.08	55.91	79.05	66.20	63.88

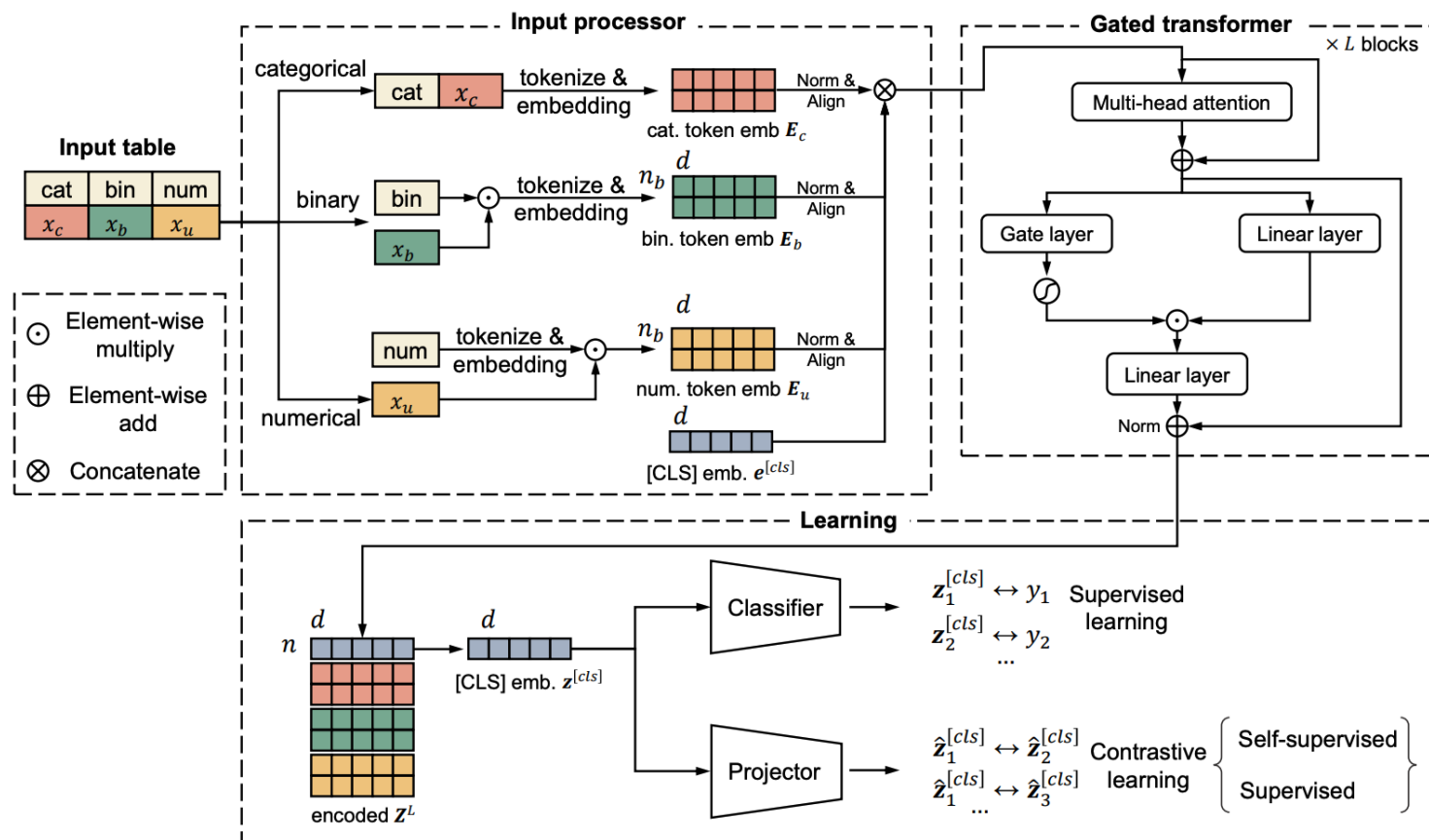
Transfer Across Different Table Schemas

- **Problem:** Most tabular models **assume a fixed set of columns**
 - Training and test tables must share the same feature schema
 - Different tables may have partially overlapping columns



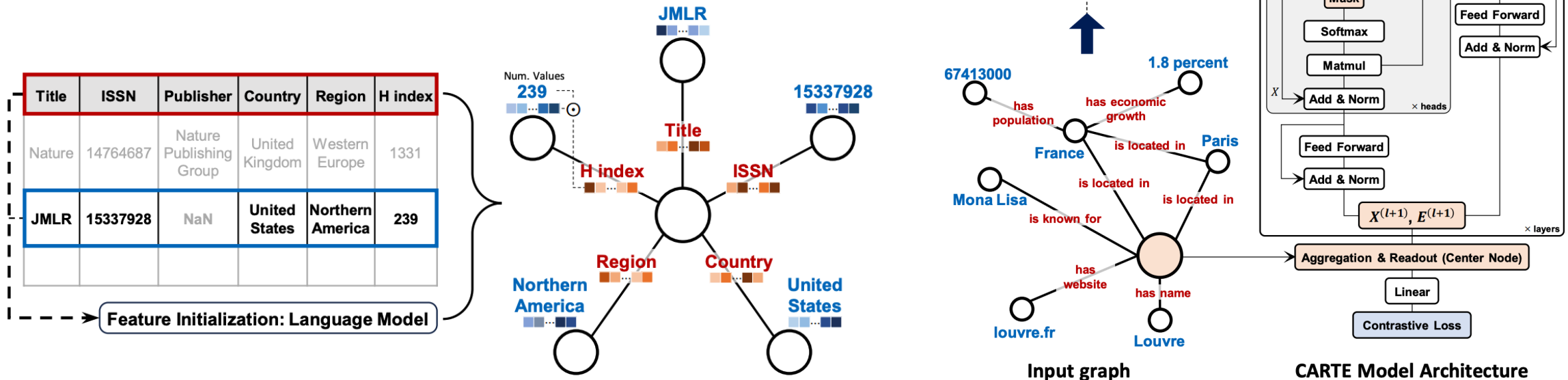
Transfer Across Different Table Schemas

- **Idea #1:** Convert tabular data into "**semantically-encoded**" tokens
 - Use the column description (e.g., column name) $\rightarrow$ transferrable



Transfer Across Different Table Schemas

- **Idea #2:** Represent each row as a graph of column names & values
 - Each row becomes a star-shaped graphlet
 - Different tables can be mapped into the same graph-based input format
 - Learn contextual entity representations from external relational knowledge



Summary: Transferable Representations

- **Goal:** Reuse tabular knowledge across limited-label tasks and heterogeneous tables
 - Few-shot adaptation from self-generated tasks
 - Transfer across different table schemas
 - Semantic transfer using column names, cell values, or external knowledge
- **Beyond Current Approaches**
 - Language models can help interpret semantic information in tables
 - Foundation models aim to transfer across broader datasets and tasks
 - We will cover these directions in later lectures

Current methods improve transfer in specific settings, motivating more general tabular learning frameworks

Tabular Anomaly Detection

- Anomaly detection is often **label-scarce**
 - Abnormal samples are rare and expensive to label
 - Many settings only provide normal training data
- **Focus on modeling what is normal, rather than directly classifying what is abnormal**
 - Normal samples should exhibit consistent patterns
 - Anomalies should be distinguishable from normal samples in the learned representation space

Key Question

How can we learn normality without explicit anomaly labels?

Two Views of Normality Learning

- **Reconstruction-based view**

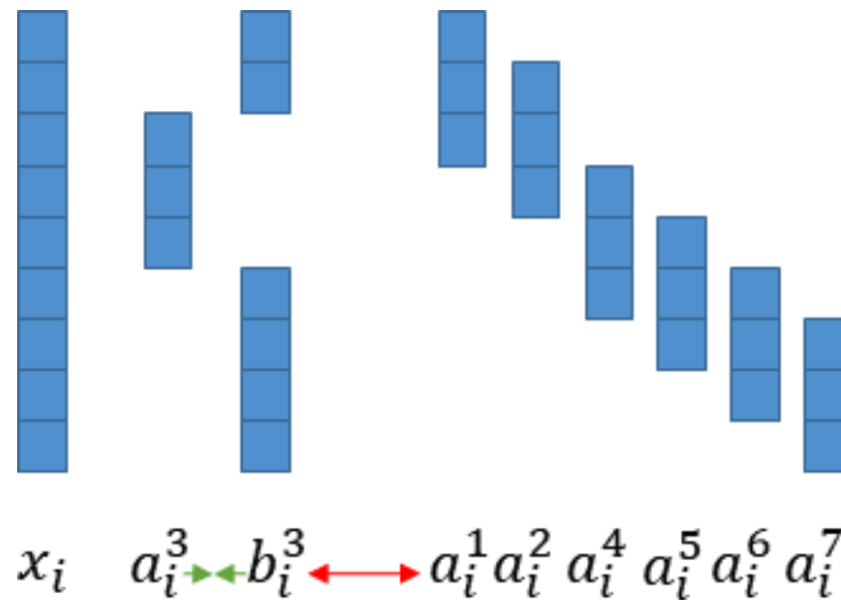
- Normal samples should be predictable from partial information
- Anomalies violate learned feature correlations
- **Anomaly score**: high reconstruction error

- **Representation-based view**

- Normal samples should lie close to a learned normal structure
- Anomalies deviate from the normal representation space
- **Anomaly score**: large distance from the normal manifold

ICL: Internal Contrastive Learning

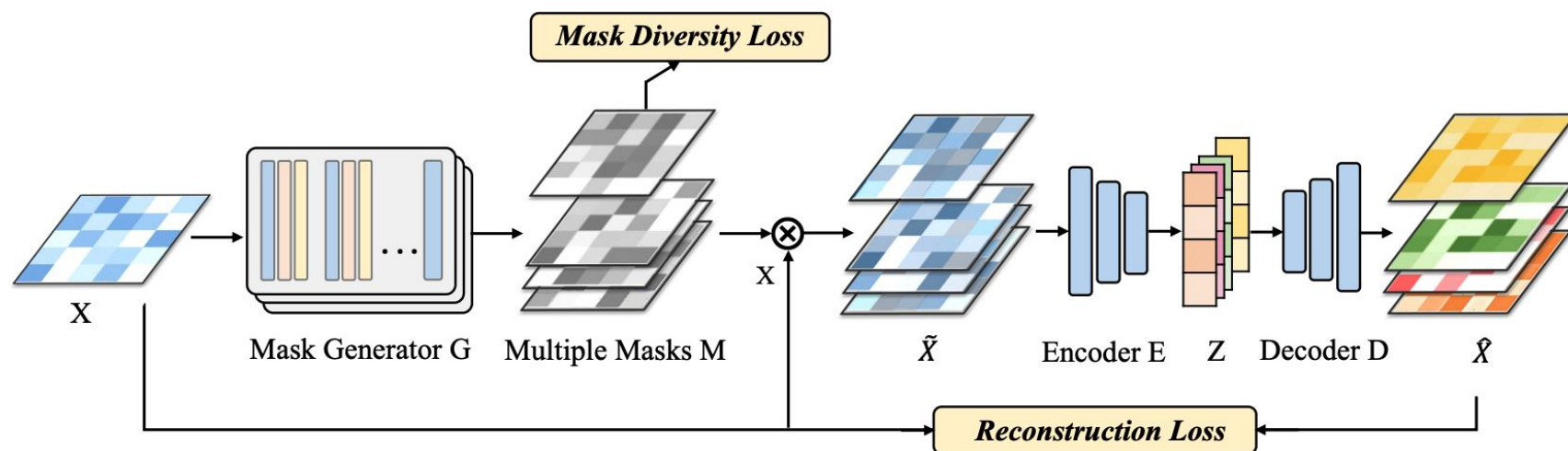
- **Key Idea:** Learn normality by matching one part of a sample with the rest of the same sample
 - Split each sample into a masked part and its complement
 - Learn two mappings that maximize agreement between matching parts
 - Use the contrastive loss itself as the anomaly score



Anomaly score:
Contrastive loss

MCM: Masked Cell Modeling

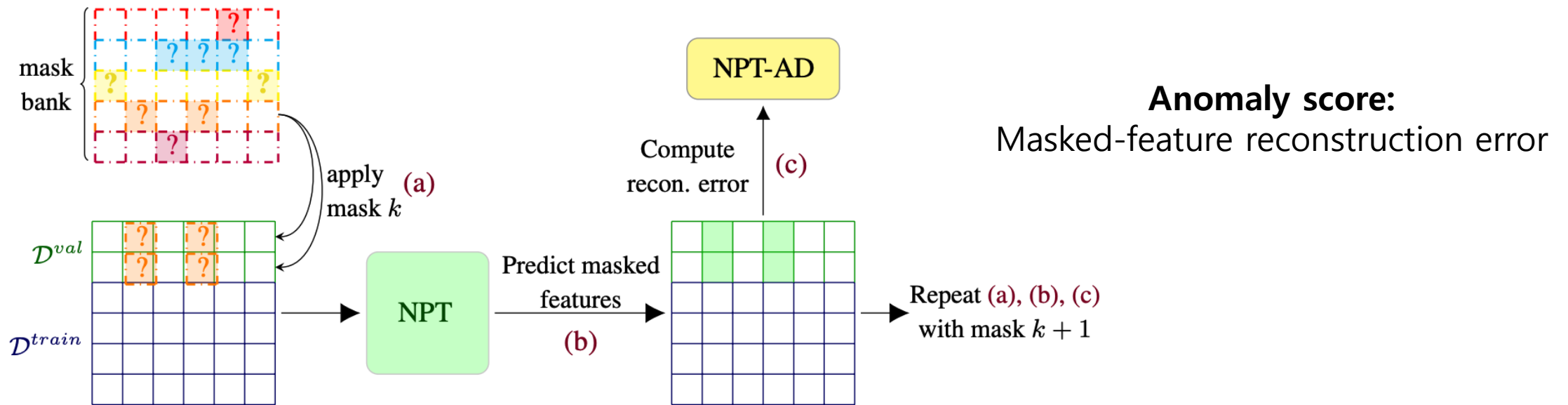
- **Key Idea:** Learn normal feature correlations through masked cell reconstruction
 - Mask cells in normal samples and reconstruct the original values
 - Use a **learnable mask generator** instead of fixed random masks
 - **Generate multiple diverse masks** to capture different normal correlations



Anomaly score:
Average reconstruction error
over multiple masked versions

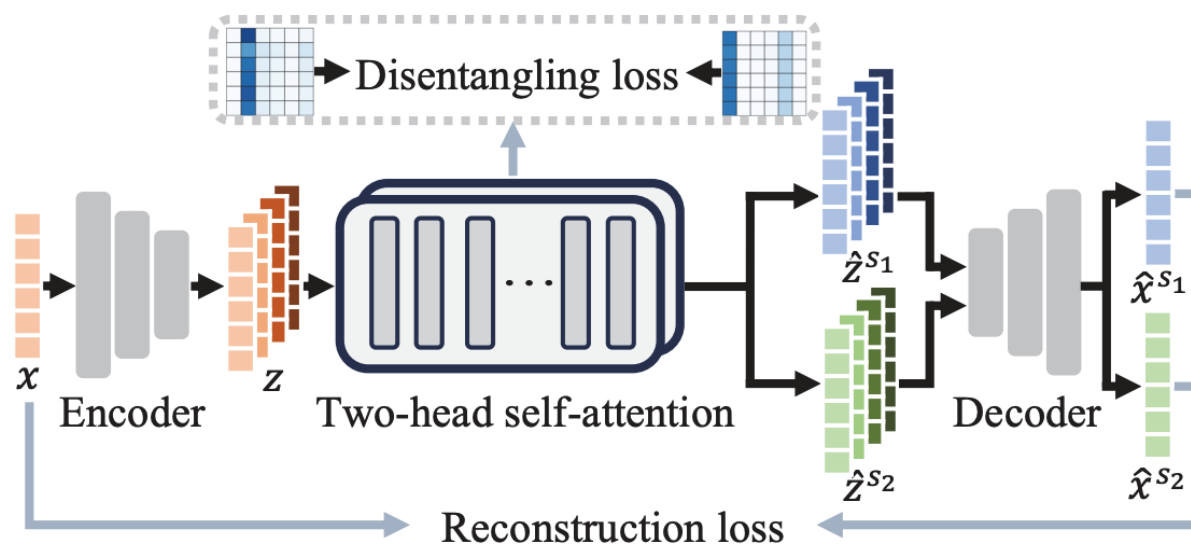
NPT-AD: Beyond Individual Input

- **Key Idea:** Reconstruct masked features using both feature-feature and sample-sample dependencies
 - Use **NPT** to combine Attention Between Attributes and Datapoints
 - Train the model to reconstruct masked features of normal samples
 - During inference, use the training set as context for reconstruction



Disent-AD: Disentangling Features

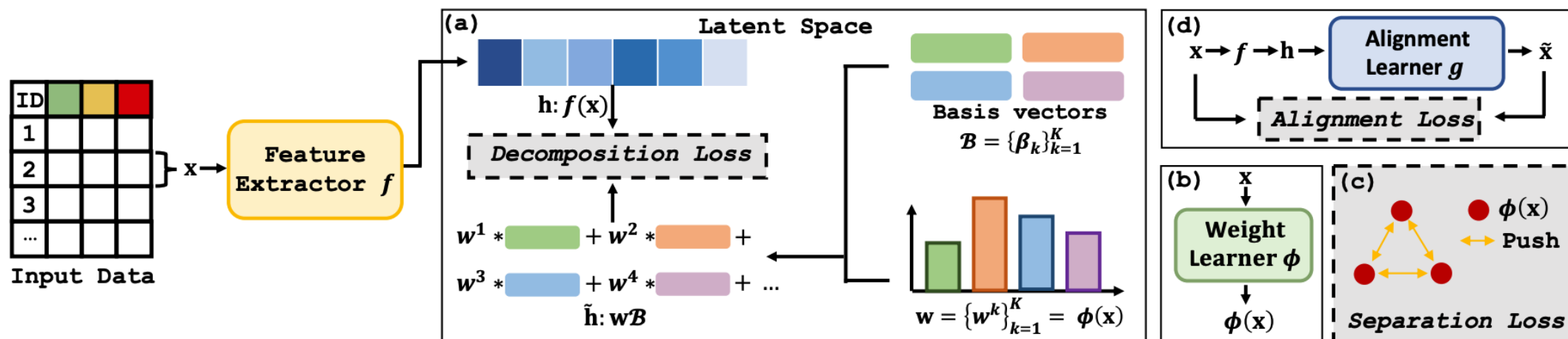
- **Key Idea:** Normal tabular attributes can be divided into non-overlapping but correlated subsets
 - Use two attention heads to extract different latent attribute subsets
 - **Reconstruct the original sample from each subset**
 - Normal samples should be recoverable from correlated subsets
 - Anomalies disrupt the correlations that make such reconstruction possible



Anomaly score:
reconstruction error

Decomposed Representation Learning

- **Key Idea:** Learn a constrained latent space in which normal representations can be decomposed using orthogonal basis vectors
 - Encode each sample into a latent representation
 - Represent normal samples as weighted sum of fixed orthogonal bases



Anomaly score: decomposition error in representation space

Summary: Tabular Anomaly Detection

Key Question

How can we learn normality without explicit anomaly labels?

Different anomaly detectors mainly differ in how they define “normal”

Method	Assumption
ICL	Normal samples have strong internal dependencies between feature subsets
MCM	Normal feature values can be recovered from informative masked contexts
NPT-AD	Normality depends on both feature-wise dependencies and sample-wise context
Disent-AD	Normal attributes can be divided into non-overlapping but correlated subsets
DRL	Normal representations can be decomposed using shared orthogonal basis structures

Tabular anomaly detection is about
choosing the right assumption for what “normal” means

Summary

- **Self-supervised learning can exploit unlabeled tables**, but general pretraining remains challenging
- **Learned representations enable broader applications:**
 - Few-shot and cross-table transfer
 - Normality modeling for anomaly detection
- The key is designing **learning signals that preserve meaningful tabular semantics**

Thank You for Your Attention!
